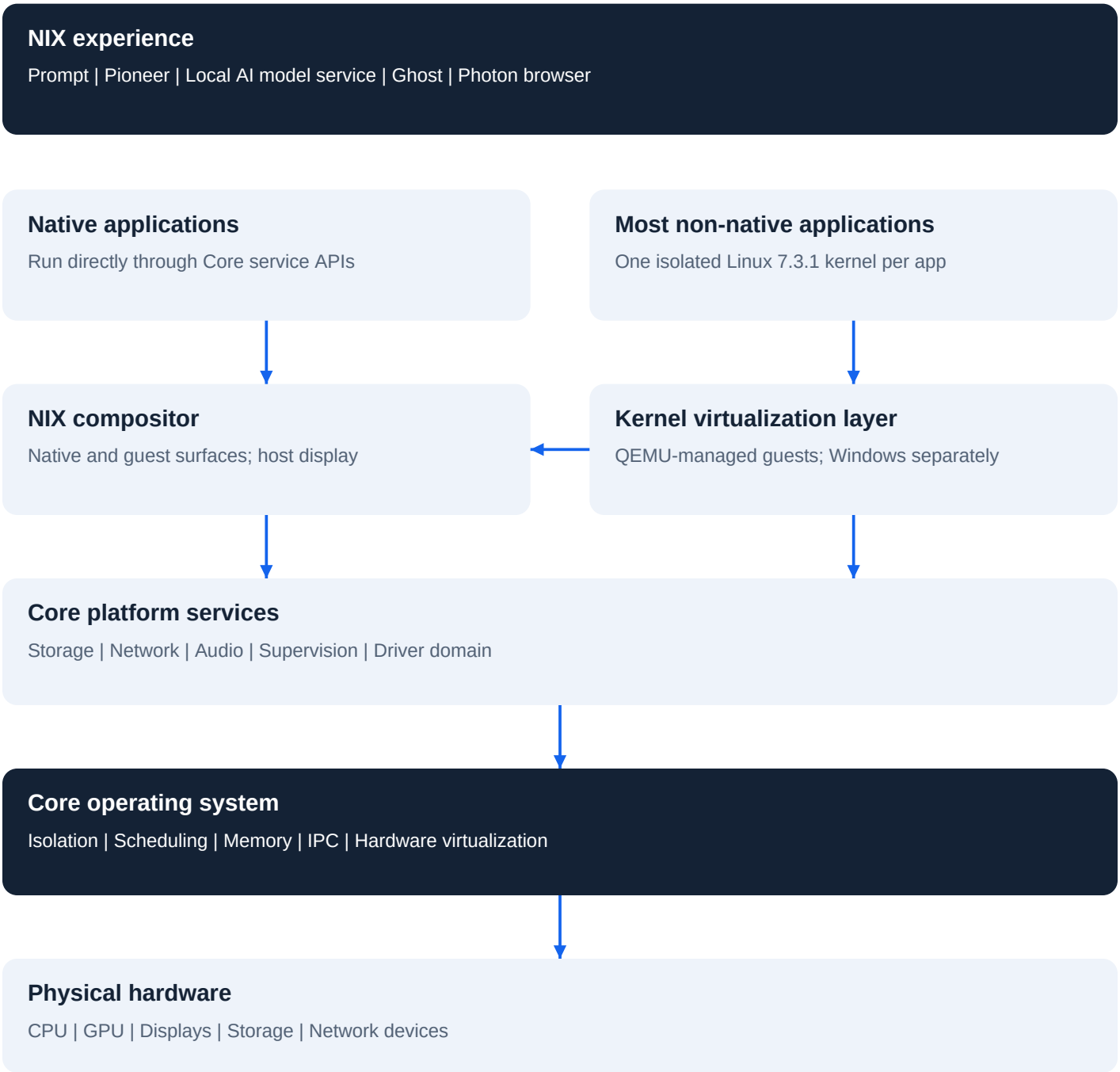
01 / SYSTEM ARCHITECTURE

A virtualization-centered OS

Core is the operating system. NIX is its prompt-led host experience.

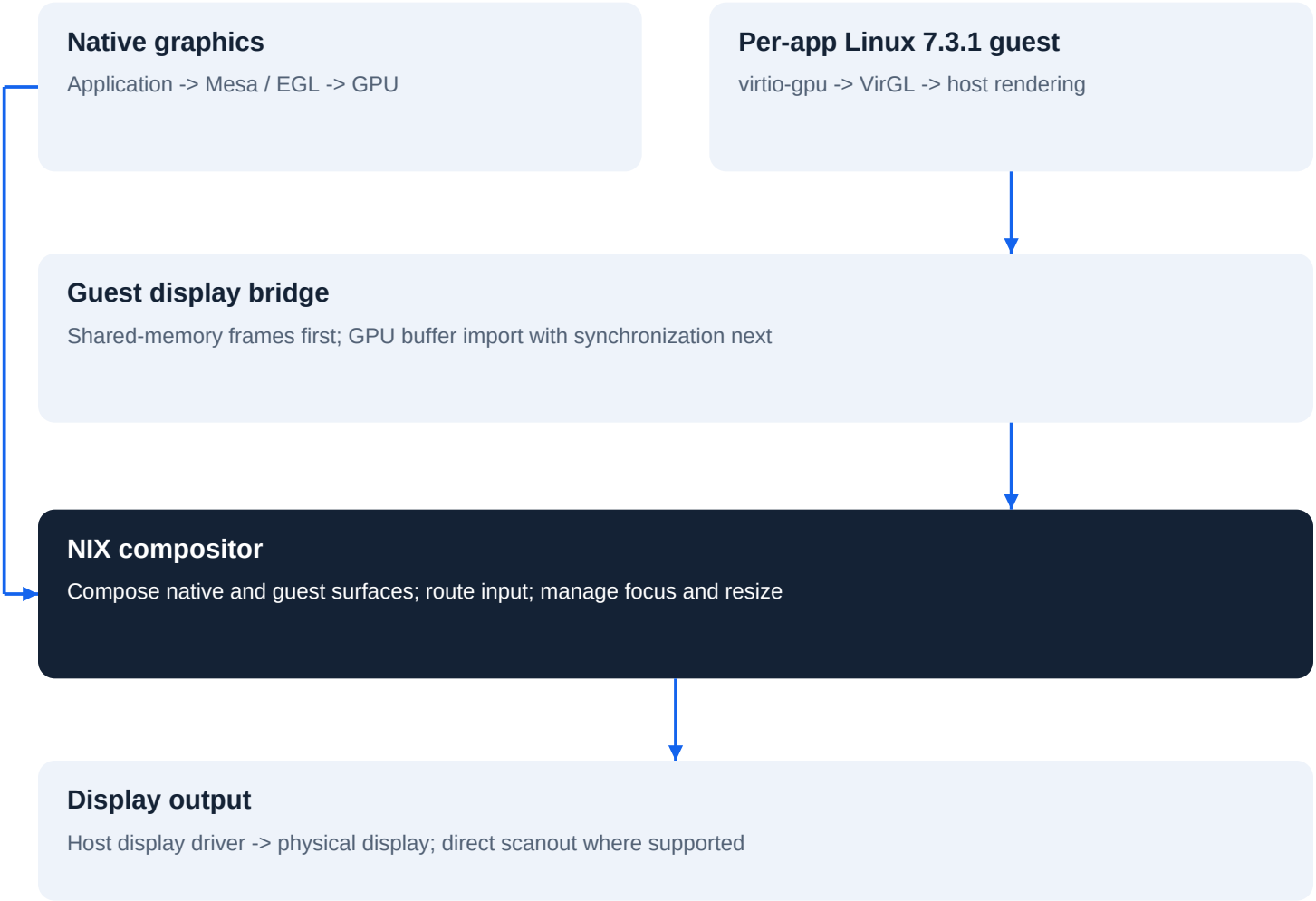


Each isolated Linux application VM owns its guest memory and kernel instance.
Native applications use Core directly; Windows workloads use Windows guests.

02 / GRAPHICS ARCHITECTURE

Native display. Shared surfaces.

Desktop OpenGL 3.0 baseline; OpenGL ES 3.0 tracked as a separate API.



Implementation contracts

Define buffer format, dimensions, stride, ownership, lifetime, and synchronization in the display protocol.

Use QEMU D-Bus display as a reference integration. Linux DMA-BUF facilities require an equivalent NIX interface or a compatible driver-domain bridge.

Keep software rendering and copied frames available for unsupported hardware. Measure copy count, frame time, and input latency.

Validate Windows display and Direct3D independently. GPU passthrough requires a distinct capture path for compositor integration.

03 / CODEX DELIVERY SEQUENCE

Execution phases 0-5

Each phase produces a reviewable change and evidence for its acceptance gate.

0 Repository audit

Read AGENTS.md and build instructions. Inventory Core OS, services, graphics, and guest virtualization. Record commit and reproducible commands.

Acceptance: Reproduce the current build and document observed behavior.

1 Architecture contracts

Define Core OS boundaries and a kernel virtualization layer. Most non-native applications each receive a Linux 7.3.1 guest. Define driver ownership and graphics interfaces.

Acceptance: Every component has an owner, interface, dependency, and failure behavior.

2 Core execution foundation

Implement isolated address spaces, scheduling, IPC, shared memory, handle ownership, and service supervision.

Acceptance: Two isolated services communicate; a failed service restarts without corrupting its peer.

3 Native display foundation

Implement display output, input, surface allocation, composition, and software rendering.

Acceptance: Two native surfaces display, resize, receive input, and recover after client failure.

4 Kernel virtualization layer

Implement QEMU host integration and per-app VM lifecycle. Pin Linux 7.3.1 guest images. Add guest resource limits, hardware acceleration, and teardown.

Acceptance: Two apps run with separate Linux kernels and memory; restarting one leaves its peer and Core running.

5 Guest display bridge

Import QEMU shared-memory frames into native surfaces. Implement resize, cursor, keyboard, and pointer routing.

Acceptance: An interactive guest appears on the host display through the NIX compositor.

04 / CODEX DELIVERY SEQUENCE

Execution phases 6-10

Each phase produces a reviewable change and evidence for its acceptance gate.

6

Native OpenGL

Integrate Mesa/EGL and a selected GPU driver path. Validate desktop OpenGL 3.0 and retain software fallback.

Acceptance: Record the renderer and pass selected API tests on named hardware.

7

Accelerated guest graphics

Integrate Linux virtio-gpu/VirGL. Add GPU buffer import, ownership, synchronization, and renderer recovery.

Acceptance: A guest 3D workload renders inside NIX; report copy count, frame timing, and recovery results.

8

Windows applications and games

Validate Windows installation, storage, network, display, input, and audio. Evaluate GPU passthrough separately.

Acceptance: Publish a compatibility matrix for named applications and games on specified hardware.

9

Prompt and application services

Connect local inference, Prompt, Pioneer, Ghost, and Photon through narrow service APIs. Enforce application launch policies.

Acceptance: A prompt launches a native or guest application and presents its surface with enforced permissions.

10

Release qualification

Implement installation, signed updates, rollback, recovery, resource limits, hardware qualification, and license inventory.

Acceptance: Install, update, recover, and execute the qualified workload suite on supported machines.

05 / CODEX TASK SPECIFICATION

Repository execution brief

Use with the caraveo/nix repository selected in Codex.

Objective

Core is the operating system; NIX provides its host experience. Most non-native applications run in their own isolated VM with a Linux 7.3.1 kernel. A kernel virtualization layer manages these environments through QEMU. Native applications run directly on Core; Windows workloads use Windows guests.

Execution requirements

1. Read AGENTS.md and repository build instructions. Reproduce the existing build and record the commit.

2. Identify Core as the operating system and NIX as the host experience. Use kernel terminology for guest kernels and the kernel virtualization layer. Review source licenses before code reuse.

3. Distinguish NIX running inside QEMU from QEMU guests running inside NIX. Specify the Core virtualization backend, per-app Linux 7.3.1 kernel instances, and graphics interfaces.

4. Follow phases 0-10 in dependency order. Resolve each acceptance gate before starting dependent implementation.

5. For every phase, identify repository-specific files, implementation tasks, verification commands, and objective evidence.

6. Keep implemented behavior separate from architecture targets. Report build results, hardware, active rendering backend, and unresolved dependencies.

7. Complete the repository audit and architecture contracts first, then implement the smallest executable phase.

Engineering references

QEMU: VirtIO GPU - qemu.org/docs/master/system/devices/virtio/virtio-gpu.html

QEMU: D-Bus display - qemu.org/docs/master/interop/dbus-display.html

Mesa: VirGL - docs.mesa3d.org/drivers/virgl.html

Linux guest baseline: kernel.org - Linux 7.3.1 (2026-10-30)

Project: github.com/caraveo/nix | core-nix.com/#architecture